Fast Track — Mock Exam 1 (Solutions)

CloudDesk scenario · full worked solutions

Prof. Dr. Christoph Weisser

HSBI

Summer Semester 2026

How to use these solutions

- Companion to `exam_1.md` — the CloudDesk parallel-form paper (100 marks, 120 min).
- **Five parts, 20 marks each**; one slide per question. The correct MCQ option is stated *and* every distractor is explained.
- **Green** = the key point / correct answer; **red** = the trap.
- Code shown is one correct solution — equivalent idioms earn full marks.
- Model marks are per sub-part; method marks apply on calculations.

Grade banding (guide)

$\geq$ 85 distinction · 70–84 strong · 50–69 pass · $<$ 50 revisit the flagged notebooks.

Outline

1. Part A — Python foundations
2. Part B — Data, pandas, viz & statistics
3. Part C — ML, evaluation & feature engineering
4. Part D — AI engineering
5. Part E — Applied & production

A1 — `int()` truncates toward zero (2 marks)

Q. What does `print(int(-3.9))` output? **Answer: B) -3**

- `int()` on a float truncates toward zero — it drops the fractional part, it does *not* round. So `-3.9` $\rightarrow$ `-3`.
- **A) -4** — that would be *rounding* (or `math.floor`); floor of -3.9 is -4, but `int` does not floor negatives.
- **C) -3.9** — `int()` always returns an `int`, never a float.
- **D) ValueError** — only strings that aren't numeric raise; a float is always castable.

Remember

`int(3.9) == 3` and `int(-3.9) == -3`. Use `round()` to round to nearest.

A2 — f-string percent format (2 marks)

Q. `rate = 0.234; f"{rate:.1%}"` **Answer: B)** `"23.4%"`

- The `%` format spec multiplies by 100 and appends `%`; `.1` keeps one decimal place: $0.234 \rightarrow 23.4\%$.
- **A)** `"0.2%"` — that ignores the $\times 100$ and just does `.1f` with a percent sign; wrong.
- **C)** `"23%"` — that is `:.0%` (zero decimals); the spec asked for one.
- **D)** `"0.234%"` — treats the number as already-a-percentage; `%` always scales.

Related specs

`{x:,}` thousands separator, `{x:.2f}` fixed decimals, `{x:>12}` right-align.

A3 — aliasing, `is` vs == (3 marks)

```
a = [1, 2, 3]
b = a # b is a second NAME for the SAME list object
b.append(4)
print(len(a)) # -> 4
```

- **Output: 4.** `b = a` does not copy; both names point at one list, so mutating through `b` is visible through `a`. *(1.5)*
- `==` tests *value* equality (same contents); `is` tests *identity* (same object in memory, same `id()`). Here `a is b` is `True`. *(1.5)*
- To get an independent copy: `b = a.copy()` / `list(a)` / `a[:]` (and `copy.deepcopy` for nested lists).

A4 — most common channel with `Counter` (4 marks)

```
from collections import Counter

counts = Counter(channels) # {'chat': 12, 'phone': 7, ...}
channel, n = counts.most_common(1)[0]
# most_common(1) -> [('chat', 12)]; [0] -> ('chat', 12); unpack
```

- `Counter(channels)` tallies each channel in one pass. *(2)*
- `.most_common(1)` returns a *list* with the single top `(value, count)` tuple; index `[0]` then unpack. *(2)*
- Accept equivalents, e.g. `max(counts, key=counts.get)` for the channel plus `counts[channel]` for the count.

A5 — mutable default argument (4 marks)

Bug: the default `batch=[]` is created once, when the function is defined, and *shared across all calls* — so appends accumulate. *(2)*

Broken

```
def log_ticket(ticket_id, batch=[]):
    batch.append(ticket_id)
    return batch
```

Fixed

```
def log_ticket(ticket_id, batch=None):
    if batch is None:
        batch = []
    batch.append(ticket_id)
    return batch
```

Use `None` as the sentinel and build a fresh list inside the body. *(2)*

A6 — SupportTicket class (5 marks)

```
class SupportTicket:
    count = 0 # class attribute (shared) -- (1)

    def __init__(self, ticket_id, csat):
        self.ticket_id = ticket_id # instance attributes -- (2)
        self.csat = csat
        SupportTicket.count += 1 # bump the shared counter -- (1)

    def is_happy(self):
        return self.csat >= 4 # method -- (1)
```

- Common slip: `self.count += 1` creates an *instance* attribute and leaves the class counter at 0 — increment `SupportTicket.count` (or `type(self).count`).
- `SupportTicket("T1", 5).is_happy()` → `True`; `SupportTicket.count` grows by 1 per instance.

Outline

1. Part A — Python foundations
2. Part B — Data, pandas, viz & statistics
3. Part C — ML, evaluation & feature engineering
4. Part D — AI engineering
5. Part E — Applied & production

B1 — boolean masks need parentheses (2 marks)

Answer: B) `df[(df["csat"] < 3) & (df["channel"] == "phone")]`

- Combine masks with the *element-wise* operator `&`, and wrap each condition in parentheses — `&` binds tighter than `<`/`==`.
- **A)** uses Python `and` — raises *"truth value of a Series is ambiguous"*.
- **C)** no parentheses: `3 & df[...]` is evaluated first ⇒ wrong / `TypeError`.
- **D)** the comma makes pandas read it as `(rows, cols)` indexing — not a mask.

B2 — seaborn error bars (2 marks)

Answer: B) the 95% confidence interval of the mean.

- By default `sns.barplot` plots the *mean* of `y` per `x` category, with error bars = the 95% CI (bootstrapped) of that mean.
- **A) ±1 std** — available via `errorbar=("sd", 1)`, but not the default.
- **C) min–max range** — seaborn never uses full range by default.
- **D) IQR** — that describes a box plot, not a bar plot's error bar.

Point

The bar shows an *estimate of the mean* with its uncertainty — overlapping CIs hint the difference may not be significant.

B3 — mean of a 0/1 column is a rate (3 marks)

`tickets.groupby("channel")["escalated"].mean()`

- Returns a **Series indexed by channel**, each value = the *mean of `escalated` within that channel*. *(1.5)*
- For a 0/1 column, mean $= \dfrac{\text{number of 1s}}{\text{number of rows}} = \dfrac{\#\text{escalated}}{\#\text{tickets}}$ — exactly the escalation rate (a proportion in $[0, 1]$). *(1.5)*
- So `0.18` for `phone` means 18% of phone tickets escalated. (This is the same trick used to read channel/plan escalation rates in the capstone.)

B4 — mean CSAT per channel, sorted (4 marks)

```
tickets.groupby("channel")["csat"].mean().sort_values()
```

- `groupby("channel")["csat"].mean()` → one mean per channel. *(2)*
- `.sort_values()` sorts *ascending* by default, so the worst channel is first. *(2)*
- Trap: `.sort_values(ascending=False)` would put the *best* first — the question asked lowest-first.

B5 — SQL: GROUP BY + HAVING (4 marks)

```sql
SELECT channel, AVG(csat) AS avg_csat
FROM tickets
GROUP BY channel
HAVING COUNT(*) > 100
ORDER BY avg_csat DESC;
```

- `GROUP BY channel` then `AVG(csat)` per group. *(1.5)*
- `HAVING` filters *groups* by an aggregate (`COUNT(*) > 100`) — `WHERE` cannot, it filters raw rows before grouping. *(1.5)*
- `ORDER BY avg_csat DESC` sorts highest-first. *(1)*

B6 — Welch t-test interpretation (5 marks)

(a) It is a **two-sample, two-sided Welch's t-test**. `equal_var=False` means it *does not assume the two groups have equal variance* (it uses the Welch–Satterthwaite d.f.) — the safer default when group sizes/spreads differ. *(2)*

(b) $p = 0.004 < 0.05$, so we reject H_0: the mean CSAT of chat and phone *differ significantly*. The negative statistic says chat's mean is the lower of the two. *(2)*

(c) Statistical $\neq$ practical significance: the **effect size** (e.g. Cohen's d) could be tiny, so the CSAT gap may be too small to justify any operational change / cost. *(1)*

Outline

1. Part A — Python foundations
2. Part B — Data, pandas, viz & statistics
3. Part C — ML, evaluation & feature engineering
4. Part D — AI engineering
5. Part E — Applied & production

C1 — what is data leakage (2 marks)

Answer: C) fitting `StandardScaler` on the whole dataset before splitting.

- That lets the scaler's `mean_`/`scale_` peek at the test rows — test information leaks into training. Fit preprocessing on *train only* (ideally inside a `Pipeline`).
- **A)** fitting on train only is exactly right — no leak.
- **B)** `stratify=y` just preserves class balance across the split — good practice.
- **D)** a `Pipeline` *prevents* leakage by re-fitting the scaler per CV fold.

C2 — `confusion_matrix().ravel()` order (2 marks)

Answer: B) `tn, fp, fn, tp`.

- For labels `[0, 1]` the matrix is row = *true*, column = *predicted*; flattening row-major gives `tn, fp, fn, tp`.
- **A)** reversed order (tp first) — wrong; that is not how `.ravel()` lays it out.
- **C)** swaps `fp` and `fn` — a classic mix-up that flips your cost sums.
- **D)** those are `classification_report` columns, not confusion-matrix cells.

Recall

FN = a real churner/fraud you *missed*; FP = a false alarm. The costs usually differ a lot.

C3 — why 99.5% accuracy is misleading (3 marks)

- The classes are **severely imbalanced** (∼0.5% fraud). A model that always predicts "not fraud" is right 99.5% of the time yet catches zero fraud — accuracy rewards the majority class and hides total failure on the class we care about. *(2)*
- Better metrics that expose it: **recall** (fraction of fraud actually caught, here 0), **PR-AUC / average precision**, or a **cost-weighted** confusion matrix. *(1)*
- Any *one* named correctly earns the mark; ROC-AUC is acceptable but PR-AUC is preferred when positives are rare.

C4 — standardise + logistic regression pipeline (4 marks)

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

model = make_pipeline(StandardScaler(),
                      LogisticRegression(max_iter=1000))
model.fit(X_train, y_train)
proba = model.predict_proba(X_test)[:, 1] # P(churn) = positive class
```

- Pipeline chains scaler → model, so the scaler is fit on train only. *(2)*
- `predict_proba(...)[:, 1]` takes column 1 = probability of the positive (churn) class — *not* `predict`, which returns hard 0/1 labels. *(2)*

C5 — expected-value offer threshold (5 marks)

(a) CLV $= m/c = 300/0.02 =$ EUR 15,000. *(1)*

(b) Break-even probability

$$p^* = \frac{C_{\text{offer}}}{s \cdot \text{CLV}} = \frac{60}{0.30 \times 15{,}000} = \frac{60}{4{,}500} = 0.0133 \ (\approx 1.3\%). \quad (2)$$

(c) Predicted churn $0.06 > p^* = 0.0133$, so the offer's expected value is *positive* — send it. Check:

$$EV = p \cdot s \cdot \text{CLV} - C_{\text{offer}} = 0.06 \times 0.30 \times 15{,}000 - 60 = 270 - 60 = \text{EUR } 210 > 0. \quad (2)$$

Key idea

The threshold is *per-customer* — a flat 0.5 cutoff is wrong; high-CLV customers justify offers at very low churn risk.

C6 — target leakage via group statistic (4 marks)

Bug: `plan_churn_rate` is computed with `transform("mean")` over all rows (train + test) using the target `churned` — so each row's feature already encodes label information from the test set. Leakage. *(2)*

```
# Fix: split FIRST, learn per-plan means on TRAIN only, map onto test.
X = data.drop(columns=["churned"]); y = data["churned"]
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2,
                                          random_state=42)
rates = y_tr.groupby(X_tr["plan"]).mean() # train-only means
X_tr["plan_churn_rate"] = X_tr["plan"].map(rates)
X_te["plan_churn_rate"] = X_te["plan"].map(rates) # reuse train stats
```

Equivalent: a per-fold `TargetEncoder` inside a `Pipeline`. *(2)*

Outline

1. Part A — Python foundations
2. Part B — Data, pandas, viz & statistics
3. Part C — ML, evaluation & feature engineering
4. Part D — AI engineering
5. Part E — Applied & production

D1 — MCP primitive controlled by the model (2 marks)

Answer: C) tools.

- MCP's three primitives map to *who is in control*: **tools** — model-controlled (the model decides to call them, like a POST endpoint with side effects).
- **A) resources** — *application*-controlled, read-only data (like a GET / file).
- **B) prompts** — *user*-controlled, invoked deliberately (like a slash command).
- **D) sampling** — not one of the three core primitives (it is a server→host request for a completion).

Mnemonic

tools = model, resources = app, prompts = user.

D2 — cosine of normalised vectors (2 marks)

Answer: B) their dot product.

- $\cos(a, b) = \dfrac{a \cdot b}{\|a\| \, \|b\|}$. If both are **L2-normalised** then $\|a\| = \|b\| = 1$, so $\cos(a, b) = a \cdot b$. This is why normalised retrieval is just `doc_emb @ q`.
- **A) Euclidean distance** — related but not equal; distance grows as similarity falls.
- **C) always 1** — only if the vectors are identical in direction.
- **D) sum of elements** — unrelated to the dot product between two vectors.

D3 — parse defensively; shape vs truth (3 marks)

(a) An LLM can return prose, markdown fences, or malformed JSON, so `json.loads` may raise `JSONDecodeError`. Wrapping it in `try/except` lets the pipeline fall back (e.g. `{"topic": "unknown"}` or a retry) instead of crashing on one bad reply. *(1.5)*

(b) Schema validation only checks the *form*: correct keys, types, ranges, non-empty. `{"order_id": "INV", "amount": 42.0}` is well-*shaped* — a non-empty string and a positive number — so it passes, even though the id is truncated and *factually wrong*. Only comparison with **ground truth** (a golden set) catches wrongness. *"Validation checks shape, not truth."* *(1.5)*

D4 — top-k by cosine similarity (4 marks)

```python
import numpy as np

def top_k(query_vec, doc_matrix, k=3):
    q = query_vec / np.linalg.norm(query_vec)
    d = doc_matrix / np.linalg.norm(doc_matrix, axis=1, keepdims=True)
    sims = d @ q # cosine sim per row
    return np.argsort(-sims)[:k] # indices, highest first
```

- Normalise the query and *each row* (`axis=1, keepdims=True`) so the dot product equals cosine similarity. *(2)*
- `np.argsort(-sims)` sorts *descending*; slice `[:k]`. *(2)*
- Also accepted: `sklearn.metrics.pairwise.cosine_similarity`.

D5 — Mean Reciprocal Rank (3 marks)

(a) Reciprocal rank of the first relevant hit per query:

$$\mathrm{RR}_1 = \tfrac{1}{1} = 1, \quad \mathrm{RR}_2 = \tfrac{1}{3} \approx 0.333, \quad \mathrm{RR}_3 = 0 \text{ (no hit)}.$$

$$\mathrm{MRR} = \frac{1 + 0.333 + 0}{3} = \frac{1.333}{3} \approx 0.444. \quad (2)$$

(b) MRR captures *how high* the first correct document is ranked (rank 1 beats rank 3), whereas a plain `retrieval@k` hit-rate only asks *whether* a correct doc is somewhere in the top-k, ignoring its position. *(1)*

D6 — LLM-as-judge biases (6 marks)

(a) Two biases (any two; 1.5 each): *(3)*

- **Verbosity / length bias** — the judge favours longer, more confident-sounding answers even when they are no more correct.
- **Position / order bias** — when comparing two answers, it tends to prefer whichever is shown *first*.

(b) Mitigations (match to the bias; 1 each): *(2)*

- Verbosity → constrain the rubric / penalise length, or use a stronger/different judge model.
- Position → swap the order and average both runs.

(c) No — 62% agreement is below the ∼80% human-agreement bar, so the judge is not yet reliable enough to gate deployments. *(1)*

Outline

1. Part A — Python foundations
2. Part B — Data, pandas, viz & statistics
3. Part C — ML, evaluation & feature engineering
4. Part D — AI engineering
5. Part E — Applied & production

E1 — cron: every 15 min on weekdays (2 marks)

Answer: C) `*/15 * * * 1-5`.

- Fields: `minute hour day-of-month month day-of-week`. `*/15` = every 15 min; `1-5` = Mon–Fri (cron uses Sunday = 0, so Mon–Fri is `1-5`).
- **A)** `0-4` — that is Sun–Thu (off by one); Friday is missed, Sunday wrongly included.
- **B)** `15 * * * 1-5` — fires *once* per hour at `:15`, not every 15 min.
- **D)** `6-7` — Saturday (6) only (7 is out of range / rarely Sunday); wrong days.

E2 — stop words drop "not" (2 marks)

Answer: B) `"not"` is an English stop word, so it is dropped before the model sees it.

- `TfidfVectorizer(stop_words="english")` removes "not", so *"not helpful"* and *"helpful"* vectorise almost identically — the negation is silently lost.
- **A)** false — logistic regression handles TF-IDF text features fine (it is the workhorse rung).
- **C)** false — TF-IDF output is L2-normalised by default.
- **D)** false — length is not the issue; the phrase vectorises, just without "not".

E3 — Docker layer ordering (3 marks)

(a) Docker caches layers top-to-bottom and invalidates every layer below the first change. Dependencies change *rarely*, code changes *constantly*. Putting `RUN pip install` above `COPY src/` means editing code does *not* invalidate the (slow) install layer — it stays cached, so rebuilds are fast. *(2)*

(b) Editing only a source file rebuilds ∼**2 layers** (the `COPY src/` and the `CMD` below it). Bumping `requirements.txt` invalidates from the `COPY requirements.txt` line downward — ∼**4 layers**, including the expensive `pip install`. *(1)*

E4 — MAE and MAPE (4 marks)

```
import numpy as np
y_true = np.array([100, 120, 90, 110])
y_pred = np.array([110, 115, 100, 105])

mae = np.abs(y_true - y_pred).mean() # 7.5
mape = (np.abs(y_true - y_pred) / np.abs(y_true)).mean() * 100 # ~7.46%
```

- MAE = mean of $|y - \hat{y}|$: $(10 + 5 + 10 + 5)/4 = 7.5$. *(2)*
- MAPE = mean of $|y - \hat{y}|/|y|$, $\times 100$:
  $(0.10 + 0.0417 + 0.1111 + 0.0455)/4 \times 100 \approx 7.46\%$. *(2)*
- Note: MAPE divides by the actuals, so it blows up if any actual is near 0 — prefer MAE when zeros are possible.

E5 — NoneType has no .get_text (4 marks)

Error: `AttributeError: 'NoneType' object has no attribute 'get_text'.` `select_one(".priority")` returns `None` when a card has no such element, and `None.get_text(...)` then crashes. *(2)*

```
badge = soup.select_one(".priority")
priority = badge.get_text(strip=True) if badge else "normal"
```

- Capture the node, test for `None`, then read it. *(2)*
- Defensive parsing turns a 3 a.m. crash into a sane default you can monitor.

E6 — costed decision rule (5 marks)

(a) Total cost from the confusion matrix:

$$\text{cost} = \text{FN} \cdot C_{\text{miss}} + \text{FP} \cdot C_{\text{false}} = 15 \cdot 40 + 40 \cdot 20 = 600 + 800 = \text{EUR } 1{,}400. \quad (2)$$

(b) *Escalate nothing* $\Rightarrow$ every real escalation is an FN (huge miss cost); *escalate everything* $\Rightarrow$ a flood of FPs. Instead, sweep the threshold (`np.linspace`), price each confusion matrix with the formula above, and pick the threshold with `np.argmin(costs)`. *(2)*

(c) The headline is the **euro cost / saving of the rule** — "the number that leaves the room." AUC is a *ranking-quality* metric, not a decision, so it can't be spent or budgeted. *(1)*

Mark summary

Part	Coverage	Marks
A	Python foundations (NB 1–5)	20
B	Data, pandas, viz & statistics (NB 6–9)	20
C	ML, evaluation & feature engineering (NB 8, 16, 17)	20
D	AI engineering: LLM, RAG, tools/agents, MCP, doc AI (NB 10–14, 18)	20
E	Applied & production: time series, NLP, deploy, scraping, capstone	20
Total — 30 questions		**100**

End of solutions — CloudDesk Mock Exam 1.